Urbana Cult

DESARROLLO WEB BACKEND · ENTREGA FINAL

Documentación Parte 3

Sistema de gestión de salas culturales, venta de entradas, membresías y control de acceso con QR.

CASO

#3 · Urbana Cult

EMPRESA DE DESARROLLO

Sintaxia · Grupo 19

INTEGRANTES

Diego Rodríguez · Brian Lavandera · Sergio Vargas · Cristian Villagra

Índice

1. Enlaces de Entrega (Despliegue, Repositorio, Drive y Video)
2. Fuentes y Bibliografía Utilizadas
3. Introducción y Alcance de la Entrega Final
4. Objetivos del Proyecto
5. Matriz de Tecnologías (Evolución Entregas 1, 2 y 3)
6. Integrantes, Roles y Responsabilidades
7. Descripción Funcional y Matriz de Permisos por Rol
8. Módulos del Proyecto y Justificación de Decisiones Técnicas
9. Comunicación en Tiempo Real e Inteligencia Artificial
10. Persistencia de Datos en MongoDB Atlas y Diagramas (Casos de Uso, Clases, DER y Secuencia)
11. Arquitectura MVC, Middlewares y Procesamiento de Peticiones
12. Trazabilidad de Requerimientos (RF / RNF) y Documentación de Pruebas
13. Historias de Usuario y Planificación
14. Mantenimiento, Evolución y Uso Consciente de Inteligencia Artificial
15. Conclusión Final del Cuatrimestre

1 Enlaces de Entrega

APLICACIÓN DESPLEGADA EN LÍNEA (RENDER - PRODUCCIÓN):
https://urbana-cult.onrender.com

REPOSITORIO GITHUB (3° ENTREGA FINAL):
https://github.com/brianlavandera1/tp2

CARPETA DE GOOGLE DRIVE (DOCUMENTACIÓN Y ENTREGAS PREVIAS/ACTUAL):
https://drive.google.com/drive/folders/1FJnjrQ9gMjxc1aAcUBWMUMIkWc-9bSrL?usp=drive_link

TRAZABILIDAD HISTÓRICA (ENTREGAS 1 Y 2):
Repositorio inicial: https://github.com/Crrisst/TP1-BackEnd/ · Video Parte 1 · Video Parte 2

LINK AL VIDEO DE DEFENSA (COLOQUIO GRUPAL - ENTREGA FINAL):
[Insertar enlace de Google Drive / YouTube del video final antes de subir al Campus]

2 Fuentes y Bibliografía Utilizadas

Material oficial de la cátedra y entregas previas: Mapa de Contenidos de *Desarrollo Web Backend* (**Bloques 1 al 11.2**) y cuadernillos teóricos **Módulos M1 a M13** del Aula Virtual (`aulasvirtuales.bue.edu.ar`): *Introducción a Node.js y Express* (Bloques 1–2), *Patrón MVC, POO y Persistencia con FS* (Bloques 3–4), *Middlewares y Motor de Plantillas Pug* (Bloques 5.1–5.2), *Asincronía (Callbacks, Promises, async/await), HTTP y Estados* (Bloques 6.1–6.3), *MongoDB, Mongoose y Operadores de Consulta* (Bloques 7.1–7.3), *Seguridad, Modelo RBAC y Hashing* (Bloque 8.0), *APIs RESTful, MongoDB Atlas y Despliegue en Render* (Bloque 9.0), y *Autenticación con Sesiones, Cookies y JSON Web Tokens — RFC 7519* (Bloques 10.0–10.2), y *WebSockets con Socket.IO e integración de Gemini* (material D12 y Bloque 11). Asimismo, se tomaron como base los artefactos de *Ingeniería de Software* para el Caso #3

(Urbana Cult) y las entregas previas *DOCUMENTACIÓN - Parte 1* y *DOCUMENTACIÓN - Parte 2* del equipo Sintaxia.

Documentación técnica oficial consultada: Node.js v20+ Documentation (nodejs.org/docs); Express 5 API Reference (expressjs.com); Mongoose ODM v9 Docs — Schemas, Populate, Plugins & Transactions (mongoosejs.com/docs); MongoDB Manual — Query & Projection Operators y Atlas Cloud (mongodb.com/docs); RFC 7519 JSON Web Token & librería `jsonwebtoken` (jwt.io); OWASP Session Management & CSRF Prevention Cheat Sheets (cheatsheetseries.owasp.org); Helmet.js Security Headers (helmetjs.github.io); Pug Template Engine (pugjs.org); Render Web Services Deployment Guide (render.com/docs); Socket.IO v4 Documentation (socket.io/docs/v4); Gemini API Documentation (ai.google.dev/gemini-api/docs).

3 Introducción y Alcance de la Entrega Final

En esta tercera instancia final de la materia *Desarrollo Web Backend*, la consultora estudiantil **Sintaxia (Grupo 19)** consolidó la versión productiva de **Urbana Cult**, un sistema integral para la gestión de salas culturales, programación de espectáculos, membresías de beneficios, boletería presencial, venta online de entradas y validación de accesos mediante códigos QR.

Cumpliendo íntegramente la hoja de ruta planteada al cierre de la segunda entrega, el proyecto evolucionó desde un Producto Mínimo Viable local con sesiones en memoria hacia una arquitectura robusta desplegada en la nube (**Render + MongoDB Atlas**). En esta etapa se incorporó encriptación irreversible de contraseñas con `bcrypt`, persistencia de sesiones en base de datos con `connect-mongo`, una API JSON protegida mediante **JSON Web Tokens (JWT)** firmados digitalmente, control de acceso granular de cuatro roles operativos más protección de Superadministrador, transacciones atómicas para evitar sobreventa de localidades, borrado lógico auditable en todas las colecciones maestras, bitácora de actividad de usuarios, exportación administrativa a planillas Excel, comunicación en tiempo real con **Socket.IO** y un asistente virtual con **Gemini**.

Alcance definido: El sistema cubre el ciclo comercial completo de un centro cultural: descubrimiento público de la cartelera con filtros sin recarga, registro validado en tres capas, contratación de planes de membresía (Estándar, Premium y VIP), carrito y cobro online simulado, venta física en ventanilla con ticket imprimible, acreditación en puerta por lectura de QR, cancelación con restitución automática de cupos y tableros gerenciales de recaudación neta, además de asistencia en tiempo real para clientes y visitantes. Quedan fuera del alcance la integración con pasarelas bancarias reales y la numeración individual de butacas por plano gráfico.

4 Objetivos del Proyecto

Objetivos generales

- ✓ Mejorar el software desarrollado en las entregas anteriores a partir de una consigna real, aplicando lo trabajado en *Ingeniería de Software*.
- ✓ Integrar un equipo de proyecto con roles especializados y documentar qué hizo cada integrante.
- ✓ Llevar la aplicación a un entorno real, con despliegue en la nube y base de datos en MongoDB Atlas.

Objetivos específicos

- ✓ Implementar una arquitectura modular MVC con rutas, controladores, middlewares propios y manejo centralizado de errores.

- ✓ Modelar los datos en MongoDB con Mongoose: entidades, relaciones y validaciones, usando programación asincrónica con `async/await`.
- ✓ Incorporar autenticación con sesiones y JWT, y autorización por roles.
- ✓ Sumar comunicación en tiempo real con Socket.IO y un asistente basado en Gemini.
- ✓ Corregir errores y mejorar seguridad, rendimiento y escalabilidad respecto de la entrega anterior.
- ✓ Documentar requerimientos, diagramas y pruebas, y automatizar las pruebas críticas del sistema.

5 Matriz de Tecnologías (Evolución Entregas 1, 2 y 3)

La siguiente matriz resume la progresión tecnológica real del proyecto a lo largo de las tres instancias evaluativas de la cursada:

Eje / Capa Técnica	1° Entrega (MVC + POO + FS)	2° Entrega (PMV + MongoDB)	3° Entrega (Cloud, RBAC & JWT)
Servidor y Arquitectura	Node.js + Express, patrón **MVC** inicial, **POO** en dominio (herencia `Cliente` / `Administrador`), logger HTTP y middleware global de errores.	MVC consolidado (`routes`, `controllers`, `models`, `views`) y programación asincrónica con `async/await`.	Express 5 modularizado con `config/`, `middlewares/` (11 módulos especializados) y `utils/` reutilizables.
Persistencia de Datos	Archivos planos JSON en `/data/` (`usuarios.json`, `eventos.json`, `salas.json`) mediante módulo nativo `fs`.	MongoDB local + ODM Mongoose (4 esquemas base: `Usuario`, `Sala`, `Evento`, `Entrada`).	**MongoDB Atlas** (Cloud) + respaldo local, **8 esquemas**, transacciones, operadores (`$gte`, `$inc`, `$or`, `$elemMatch`), agregaciones y **borrado lógico**.
Autenticación y Sesiones	Sin sesiones persistentes; distinción conceptual de cuentas mediante clases POO y parámetros.	`express-session` en memoria RAM del proceso; contraseñas en texto plano con fines académicos.	Híbrida: **Sesiones en MongoDB** (`connect-mongo`, cookies `HttpOnly` / `Secure`) + **JWT (`jsonwebtoken` HS256)** en API REST. Hash con `bcrypt`.
Autorización y Seguridad	Validaciones de datos obligatorios, formato y tipos en controladores antes de persistir en JSON.	2 roles básicos (`Cliente` y `Admin`) con middlewares por ruta y expresiones regulares en formularios.	Modelo **RBAC de 4 roles + Superadmin**, `helmet` (CSP), anti-CSRF (`verificarOrigen`), anti-fuerza bruta (`limiteLogin` HTTP 429) y triple validación.

Eje / Capa Técnica	1° Entrega (MVC + POO + FS)	2° Entrega (PMV + MongoDB)	3° Entrega (Cloud, RBAC & JWT)
Vistas y Experiencia (Front)	Motor de plantillas **Pug** inicial (layouts e iteraciones dinámicas) + respuestas JSON para API CRUD.	Catálogo público en Pug, carrito de compras en sesión, pasarela simulada y ticket con QR en cliente.	Diseño integral *Urbana Cult* responsive mobile, filtros dinámicos, fotos binarias de artistas (`multer`), membresías, boletería física y lector QR en puerta.
Tiempo Real e IA	No aplica en esta etapa.	No aplica en esta etapa.	**Socket.IO** (conteo de conectados y chat) y asistente con **Gemini** (`@google/generative-ai`).
Pruebas, Módulos y Despliegue	Ejecución local (`localhost`) y pruebas de endpoints CRUD verificadas con **Thunder Client**.	Variables de entorno con `dotenv` y pruebas funcionales integrales de flujo web.	Despliegue productivo en **Render** (HTTPS + `trust proxy`), reportes `exceljs`, `qrcode` SVG y bitácora de auditoría de actividad.

6 Integrantes, Roles y Responsabilidades

Integrante	Rol en el Equipo	Evolución y Responsabilidades en la Entrega Final
Brian Lavandera	Líder Técnico / Seguridad & Cloud	*(E1: POO en Usuarios · E2: Auth y Sesiones).* En esta entrega final lideró la configuración de entorno (`config.js`), la conexión resiliente a MongoDB Atlas (`db.js`), la autenticación dual con sesiones persistentes (`connect-mongo`) y emisión/verificación de **JWT** (`jwt.middleware.js`, `POST /auth/token`), el hash con `bcrypt`, las defensas `helmet` / CSRF / rate-limit y el despliegue productivo en **Render**.
Diego Rodríguez	Backend Developer / Post-Venta & Auditoría	*(E1: Update Salas/Eventos · E2: Modelo Entrada y QR).* Evolucionó `Entrada.js` incorporando cancelación transaccional con restitución de stock y registro en `CancelacionLog.js`, desarrolló la validación atómica de QR en puerta (`/admin/validar/:codigo`), el plugin transversal de borrado lógico (`borradoLogico.js`), la bitácora de actividad por sesión (`Actividad.js`) y la documentación técnica.
Sergio Vargas	Full-Stack Developer / UX Cliente & Carrito	*(E1: Validaciones y Thunder Client · E2: Carrito y Checkout).* Desarrolló la interfaz responsive en Pug (`estilosUrbana.pug`), la cartelera con filtros combinados sin recarga, la carga validada de fotografías de artistas con `multer` (`Entidad.js`), el sistema de membresías con descuentos automáticos (Estándar, Premium 15 %, VIP 30 %) y el checkout atómico por sector.

Integrante	Rol en el Equipo	Evolución y Responsabilidades en la Entrega Final
Cristian Villagra	Backend Developer / Back-Office, API & Reportes	*(E1: Arquitectura MVC y Pug base · E2: Panel Admin y Salas).* Consolidó el Back-Office (`/admin`) bajo modelo RBAC para Administrador, Supervisor y Lector, implementó el módulo de boletería presencial en sucursal (`boleteria.pug`), los pipelines de agregación financiera en MongoDB, la exportación de reportes a `.xlsx` (`excelController.js`) y los endpoints de la API RESTful `/entradas` .

7 Descripción Funcional y Matriz de Permisos por Rol

Portal Público y Experiencia del Cliente (B2C): Cualquier visitante puede explorar la portada, la cartelera cultural con precios por sector (General, Platea y VIP), la calculadora de ahorro de membresías y el centro de ayuda sin necesidad de iniciar sesión, y consultar en privado a un asistente con Gemini. Al autenticarse, el cliente accede a su carrito persistente, aplica automáticamente el descuento de su membresía en el checkout, descarga e imprime sus entradas con código QR único, gestiona sus datos personales y puede cancelar tickets vigentes devolviendo el cupo al espectáculo.

Panel de Gestión Operativa y Auditoría (Back-Office): El personal interno ingresa a un panel lateral adaptado estrictamente a su jerarquía organizacional. De este modo —tal como se analizó en clase respecto a la seguridad interna en empresas—, un empleado con rol de lectura o supervisión operativa no puede alterar usuarios privilegiados ni acceder a funciones fuera de su competencia, validándose cada permiso en el servidor mediante middlewares antes de ejecutar cualquier controlador.

Funcionalidad del Sistema	Cliente	Lector	Supervisor	Admin
Explorar cartelera, comprar online, gestionar membresía, perfil y cancelar entradas propias	✓	—	—	—
Consultar dashboard, listado de salas/eventos/artistas, pagos, cancelaciones e informes (y Excel)	—	✓	✓	✓
Crear, editar y dar de baja lógica salas, eventos y artistas/entidades (con fotografía)	—	—	✓	✓
Operar Boletería Física en sucursal y validar entradas con código QR en puerta	—	—	✓	✓
Auditar el registro detallado de actividad por sesión, clics e histórico de eliminados	—	—	✓	✓
Chat en vivo con Gemini (las visitas sin cuenta lo usan en privado y con límite de consultas)	✓	—	—	—
Ver en tiempo real la cantidad de usuarios conectados	—	✓	✓	✓
Gestionar cuentas de usuarios y personal interno (solo el Superadmin `admin` gestiona a otros Admins)	—	—	—	✓
Operaciones de escritura en la API JSON (`POST` , `PUT` , `DELETE /entradas`) con JWT	—	—	—	✓

8 Módulos del Proyecto y Justificación de Decisiones Técnicas

- **Autenticación Dual (Sesiones + JWT) y Seguridad:** Login web con regeneración de ID de sesión (anti *session fixation*), cookie `httpOnly` / `sameSite=lax` / `secure`, y endpoint `POST /auth/token` para clientes API mediante tokens firmados con `HS256`. Incluye bloqueo temporal ante 5 intentos fallidos en 15 minutos (`limiteLogin`).

- **Gestión de Salas, Artistas y Eventos:** Verificación matemática de aforos por sector (`General + Platea + VIP = Capacidad`), bloqueo de modificación estructural si existen funciones asociadas (HTTP 409) y carga validada por *magic bytes* de imágenes JPG/PNG/WEBP de hasta 2 MB en MongoDB.

- **Carrito, Membresías y Transacciones Atómicas:** Aplicación automática de descuentos (0 %, 15 %, 30 %), validación de medios de pago y ejecución envuelta en `conTransaccion()` (`utils/transaccion.js`): usa sesiones ACID nativas en MongoDB Atlas y compensación automática si se ejecuta sobre un MongoDB local *standalone*.

- **Boletería Presencial y Control de Acceso QR:** Venta en ventanilla con descuento en efectivo, generación de QR en formato SVG vectorial (`qrcode`) apuntando a `/admin/validar/:codigo` y marcado atómico `Completado → Usado` que impide el reingreso con el mismo ticket.

- **Auditoría Contable, Borrado Lógico y Exportación Excel:** Ninguna entidad maestra se destruye físicamente (plugin `borradoLogico.js`). Los informes financieros descuentan automáticamente las entradas canceladas y permiten descargar libros `.xlsx` formateados mediante `exceljs`.

- **Tiempo Real y Asistente con IA:** Conteo de usuarios conectados para la empresa con Socket.IO, chat en vivo entre clientes y asistente con Gemini por medio del prefijo `@gemini`, con conversación privada y límites de consultas para las visitas.

Justificación de decisiones arquitectónicas, herramientas omitidas y módulos adicionales

Decisiones alineadas a los bloques teóricos de la cátedra: Para el control de permisos se adoptó el modelo **RBAC** (*Role-Based Access Control*, Bloques 8.0 y 10.0) restringiendo roles con `enum` en Mongoose y validándolos en servidor mediante `verificarRol`. Para el resguardo de credenciales, se evolucionó del concepto de hash con *salt* visto con `PBKDF2` (Bloque 8.0) hacia **`bcrypt`** con factor de costo 10 (Bloque 10.2), que gestiona el *salt* criptográfico por usuario de forma nativa y resistente a fuerza bruta. En materia de comunicación HTTP y consultas (Bloques 6.2, 6.3, 7.2 y 7.3), las operaciones críticas emplean operadores atómicos y lógicos de MongoDB (`$gte`, `$lte`, `$or`, `$ne`, `$inc`, `$elemMatch`, `$expr`) devolviendo códigos HTTP semánticos estrictos (`200`, `201`, `302`, `400`, `401`, `403`, `404`, `409`, `429`, `500`).

Herramientas no utilizadas: No se incorporó **Passport.js** dado que el sistema emplea credenciales locales sin proveedores OAuth externos, permitiendo control directo sobre la regeneración de sesión, el bloqueo por intentos y la firma de JWT. Tampoco se habilitó el middleware **`cors`** abierto (Bloque 6.2) porque las vistas Pug se sirven desde el mismo origen del backend, reforzándose en cambio con validación estricta de cabecera `Origin` / `Referer` (`verificarOrigen`) contra ataques CSRF; ni se utilizó **`mongoose-unique-validator`** por encontrarse deprecado en Mongoose 8/9, resolviendo la unicidad mediante índices nativos `unique: true` y captura del código `11000`. Asimismo, se eligió **coexistencia de Sesiones y JWT**: cookies de sesión en MongoDB para la navegación SSR en Pug y tokens *stateless* `Bearer` (RFC 7519) para la API RESTful `/entradas`.

Módulos adicionales incorporados: Se sumaron cinco paquetes justificados por requerimientos concretos del dominio: **`helmet`** (cabeceras HTTP seguras y Content Security Policy contra XSS y clickjacking), **`connect-mongo`** (persistencia de sesiones en MongoDB Atlas para sobrevivir reinicios en Render con expiración TTL), **`multer`**

(carga en memoria y validación binaria de fotografías de artistas), `qrcode` (generación en servidor de códigos QR vectoriales en SVG para cada ticket) y `exceljs` (exportación administrativa descargable de pagos y recaudación en formato `.xlsx`). También se incorporaron `socket.io` y `@google/generative-ai`, trabajados en clase, para la comunicación en tiempo real y el asistente virtual, y `socket.io-client` solo para las pruebas automáticas.

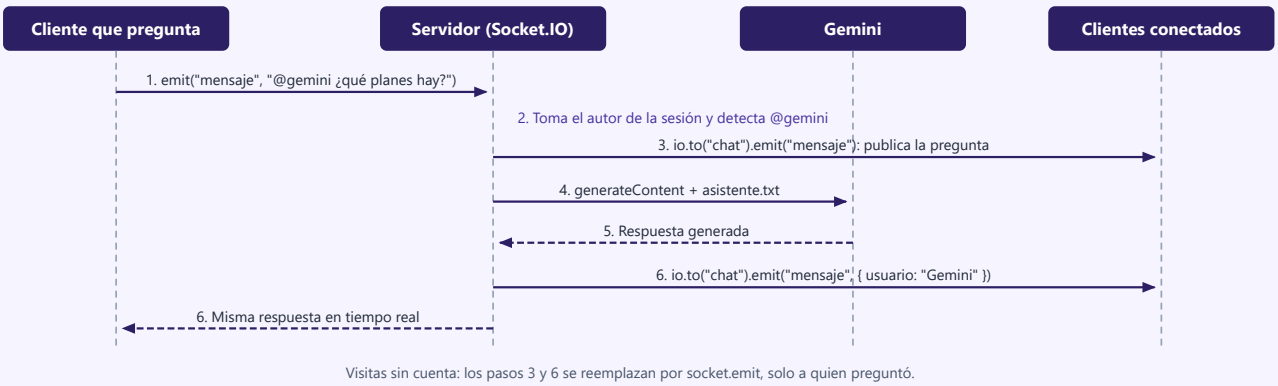
9 Comunicación en Tiempo Real e Inteligencia Artificial

La comunicación en tiempo real se implementó con **Socket.IO** sobre el servidor HTTP de Node (`http.createServer(app)`), siguiendo el módulo *D12. Websockets*. El código está en `sockets/`, separado de las rutas: `index.js` crea el servidor de sockets y comparte la sesión de Express, `conectados.js` mantiene el conteo y `chat.js` resuelve el chat.

Funcionalidad	Quién la usa	Funcionamiento
Conteo de usuarios conectados	Lector, Supervisor y Administrador	Se cuenta a todos los navegadores conectados, incluidas las visitas, y el total se emite solo a la sala `personal`. Varias pestañas cuentan como un usuario, y una espera de 4 segundos evita que el número cambie al navegar entre páginas.
Chat en vivo con Gemini	Clientes con sesión	Sala compartida: los mensajes llegan a todos al instante. El autor se obtiene de la sesión del servidor, por lo que nadie puede escribir con el nombre de otro. El mensaje con prefijo `@gemini` se envía a la IA y la respuesta se publica para todos.
Asistente para visitas	Visitantes sin cuenta	Conversación privada: preguntas y respuestas se envían solo a ese socket. Límite de 10 consultas por hora por visitante y 100 por hora entre todas las visitas, para cuidar el cupo de la clave. Al llegar al límite se invita a crear una cuenta.

Integración con Gemini: `utils/gemini.js` inicializa el modelo una sola vez con `@google/generative-ai` y la clave `GEMINI_API_KEY`. El comportamiento del asistente se define en el archivo de texto `config/asistente.txt`, con la información del centro de ayuda, y se le indica que no invente datos. El material de clase usa `gemini-2.5-flash`, pero Google ya no lo ofrece a claves nuevas, por lo que se configuró `gemini-flash-lite-latest`, modificable con la variable `GEMINI_MODEL`. Si la IA falla, el usuario recibe un aviso y el servidor sigue funcionando.

DIAGRAMA DE SECUENCIA · CONSULTA A GEMINI DESDE EL CHAT



10 Persistencia de Datos en MongoDB Atlas y Diagramas

La capa de datos se aloja en un clúster cloud de **MongoDB Atlas** e implementa ocho modelos Mongoose (más la colección automática `sesiones` administrada por `connect-mongo`):

Modelo Mongoose	Atributos Principales y Validaciones de Esquema	Relaciones NoSQL y Reglas
Usuario `Usuario.js`	`nombreUsuario` (unique), `email` (unique, regex), `password` (hash bcrypt), `nombre` y `apellido` (solo letras), `dni` (7–8 dígitos), `telefono` (10–15 dígitos), `fechaNacimiento` (+18 años), `rol` (enum 4 roles), `tipoCliente`, `porcentajeDescuento`.	Referenciado por `Transaccion`, `Entrada`, `CancelacionLog` y `Actividad`. Incluye plugin `borradoLogico`.
Sala `Sala.js`	`nombre` (unique), `capacidad` (entero ≥ 1), subdocumento `zonas` (`General`, `Platea`, `VIP` enteros ≥ 0), `ubicacion`, `disponible`. Hook `pre('validate')` recalcula el total.	Relación 1:N con `Evento`. Impide editar zonas o eliminar si posee eventos asociados (HTTP 409).
Entidad `Entidad.js`	`nombre` (unique), `foto` binaria (`Buffer` + `contentType` JPG/PNG/WEBP con `select: false`), `fotoActualizada`, `fotoCredito` (autor y licencia CC).	Relación 1:N con `Evento`. Servida bajo demanda por `GET /media/artistas/:id`.
Evento `Evento.js`	`nombre`, `descripcion`, `categoria` (enum 7 rubros), `fecha`, `hora`, `precio` (General), `precioPlatea`, `precioVIP`, `entradasDisponibles` (min: 0), `vendidosPorZona`.	Referencias `ObjectId` hacia `Sala` y `Entidad`. Métodos atómicos `reservarZona` y `liberarZona`.
Transaccion `Transaccion.js`	`comprobanteId` (unique), `items[]` embebidos (`eventoId`, `tipoEntrada`, `cantidad`, `cantidadCancelada`, `precio`), `subtotal`, `descuentoAplicado`, `total`, `medioPago`, `canal` (`Online` | `Sucursal`).	Referencia a `Usuario` (`cliente`) y a `Evento` en cada ítem. Base de las agregaciones contables.
Entrada `Entrada.js`	`codigoBoleto` (unique `TICK-XXXXXXXXXX`), `sector`, `cantidad`, `precio`, `fila`, `asiento`, `estado` (`Completado` | `Usado` | `Cancelada`), `usadaEn`, `validadaPor`.	Referencias triples hacia `Usuario`, `Evento` y `Transaccion`.
CancelacionLog `CancelacionLog.js`	`nombreCliente`, `eventoNombre`, `codigoBoleto`, `cantidad`, `fechaCancelacion`.	Referencias a `Usuario` y `Evento` para trazabilidad de devoluciones.
Actividad `Actividad.js`	`nombreUsuario`, `rol`, `sesionId`, `tipo` (`sesion` | `navegacion` | `accion` | `click`), `accion`, `pagina`, `metodo`, `estado`, `ip`, `fecha`.	Referencia opcional a `Usuario`. Posee 3 índices compuestos por fecha descendente.

DIAGRAMA DE CASOS DE USO

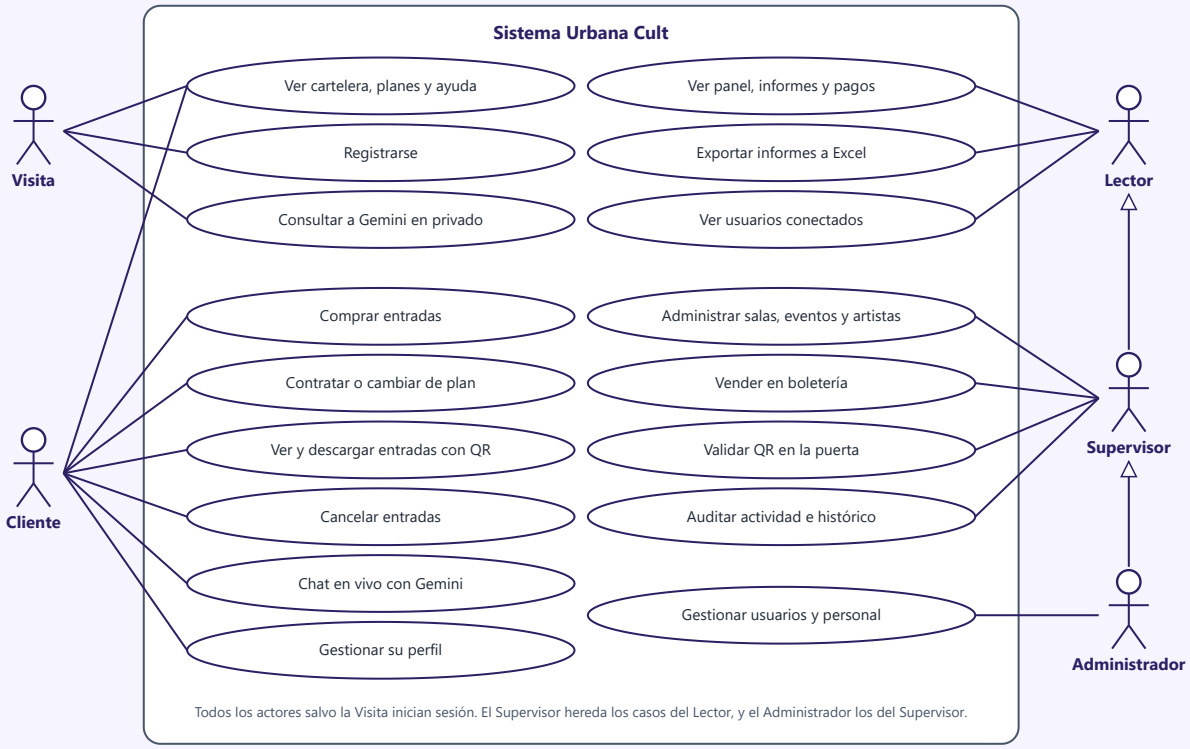
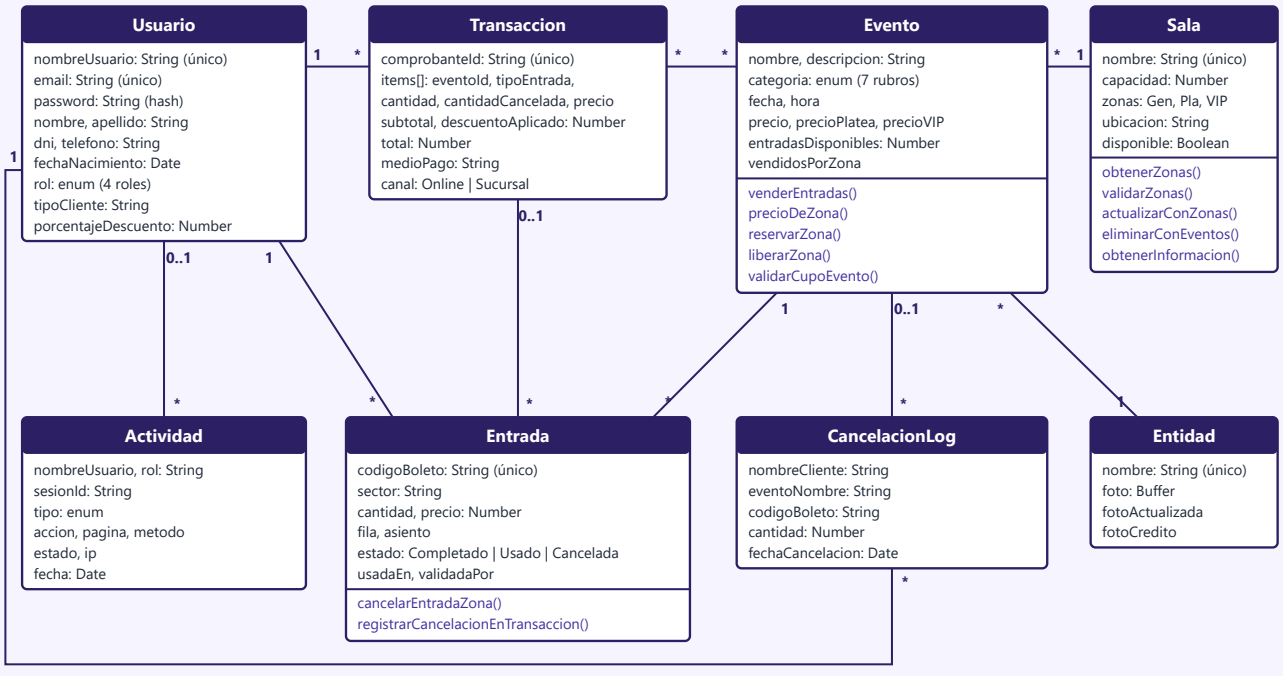


DIAGRAMA DE CLASES (MODELOS MONGOOSE)



Las relaciones son referencias ObjectId de Mongoose. Usuario, Sala, Entidad y Evento incluyen el plugin borradoLogico: eliminarLogico().

DIAGRAMA ENTIDAD-RELACIÓN (MODELO DE DATOS NOSQL EN MONGODB)

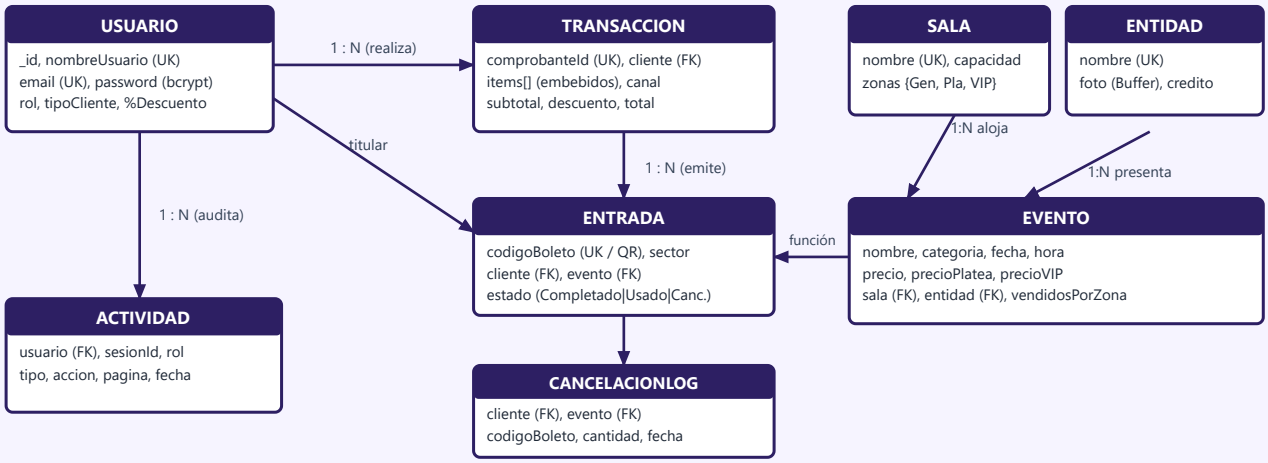


DIAGRAMA DE SECUENCIA · AUTENTICACIÓN Y AUTORIZACIÓN JWT EN API REST (/ENTRADAS)



11 Arquitectura MVC, Middlewares y Procesamiento de Peticiones

La aplicación organiza responsabilidades bajo el patrón **Modelo–Vista–Controlador (MVC)**, aislando la configuración de base de datos, las reglas reutilizables de negocio (`utils/`) y once middlewares transversales:

```
├── config/       → config.js (variables .env y secretos) · db.js (MongoDB Atlas + respaldo local) · asistente.txt (contexto de Gemini)
├── models/       → 8 esquemas Mongoose + plugins/borradoLogico.js
├── controllers/  → 10 controladores (auth, cliente, carrito, admin, eventos, salas, entidades, entradas, actividad, excel)
├── middlewares/  → authMiddleware, jwt, limiteLogin, origen (CSRF), validaciones, validateSala, subirFoto, actividad, logger, asyncHandler, errorHandler (+ index.js que los reúne)
├── routes/       → 9 enrutadores Express modulares montados en index.js
├── sockets/      → index.js (Socket.IO y sesión) · conectados.js (conteo) · chat.js (chat y visitas)
├── utils/        → transaccion.js (ACID/compensación), paginar.js, qr.js, excel.js, escapeRegex.js, gemini.js
├── tests/        → pruebas automáticas (npm test), siempre sobre una base local de pruebas
└── views/        → Plantillas Pug (layouts Admin/Cliente/Acceso, partials reutilizables y vistas por rol)
```

Ruta / Prefijo y Método	Cadena de Middlewares y Comportamiento del Controlador
`/socket.io` (WebSocket)	Verifica que la conexión venga del propio sitio y lee la sesión al conectarse. Según el rol, habilita el chat, la conversación privada o el conteo de conectados.
`POST /auth/login` `GET /auth/logout`	`limiteLogin` → `procesarLogin` verifica hash `bcrypt`, regenera sesión en MongoDB y redirige según rol. `/logout` destruye la sesión y limpia la cookie `connect.sid`.
`POST /auth/token`	`limiteLogin` → `emitirToken` valida credenciales y firma un **JWT** (HS256, audiencia, emisor y expiración `1h`) sin exponer datos sensibles en el payload.
`GET|POST|PUT|DELETE /entradas`	API JSON: `autenticarToken` → `verificarAutenticacion` → `verificarRol(['Administrador'])` (en escritura). Lista tickets propios o emite/anula entradas ajustando stock.
`GET /cliente/comprar` `POST /cliente/carrito/procesar-pago`	Cartelera pública con filtros combinados. El pago requiere sesión activa, aplica descuento de membresía y ejecuta `conTransaccion()` reservando zona y emitiendo tickets con QR.
`POST /cliente/cancelar-entrada/:id`	`verificarRol(['Cliente'])` → `Entrada.cancelarEntradaZona` cambia estado a `Cancelada`, devuelve el cupo al sector, descuenta de `Transaccion` y graba `CancelacionLog`.
`GET|POST /admin/boleteria` `GET|POST /admin/validar/:codigo`	`verificarRol(['Administrador','Supervisor'])`. Venta presencial en sucursal con ticket imprimible, y escaneo de QR en puerta con marcado atómico `Usado`.
`GET /admin/informe` · `/pagos` `GET /admin/informe/excel`	`verificarRol(['Administrador','Supervisor','Lector'])`. Pipelines de agregación MongoDB sobre recaudación neta y descarga directa en formato `.xlsx`.

12 Trazabilidad de Requerimientos (RF / RNF) y Documentación de Pruebas

Integrando los artefactos de *Ingeniería de Software*, se detalla la trazabilidad de requerimientos clave y la matriz de pruebas funcionales y de seguridad ejecutadas sobre el sistema desplegado:

Código	Descripción del Requerimiento	Implementación Técnica Verificada
RF-01 / RF-09	Gestión de salas por sectores, artistas con foto y eventos con cupo controlado	Modelos `Sala`, `Entidad` y `Evento` con validación de suma de zonas, carga binaria con `multer` y control de capacidad máxima.
RF-06 / RF-12	Venta online (con membresías) y venta presencial en boletería física	Controladores `carritoController` y `adminController.procesarVentaFisica` unificados bajo `Transaccion` (canal `Online` / `Sucursal`).
RF-07 / RF-14	Emisión de entradas con QR y validación única en puerta	Generación SVG con `utils/qr.js` y validación atómica en `POST /admin/validar/:codigo` que impide doble acreditación.

Código	Descripción del Requerimiento	Implementación Técnica Verificada
RF-15 / RF-16	Cancelación con devolución de stock y reportes gerenciales exportables	Método transaccional `cancelarEntradaZona`, agregaciones netas en `entradasController` y exportación con `excelController`.
RF-TR1	Mostrar a la empresa la cantidad de usuarios conectados en tiempo real	Módulo `sockets/conectados.js` con emisión exclusiva a la sala `personal`.
RF-TR2	Chat en tiempo real con asistente virtual que responde consultas	Módulos `sockets/chat.js` y `utils/gemini.js`, con contexto en `config/asistente.txt`.
RNF-01	Seguridad de acceso, sesiones, API stateless y protección de cabeceras	Hash `bcrypt`, cookies seguras, JWT HS256, `helmet` (CSP), anti-CSRF (`verificarOrigen`) y rate-limit (`limiteLogin`).
RNF-02	Integridad transaccional y consistencia ante compras concurrentes	Filtros condicionales atómicos (`$gte` / `$inc`) en MongoDB y wrapper `conTransaccion()` para todo el flujo de compra/cancelación.
RNF-03	Rendimiento y escalabilidad en auditoría de alto volumen	Consultas `lean()` paginadas, acotación temporal e índices compuestos en `Actividad` (respuesta reducida de ~1,4 s a ~0,4 s con 200k registros).

Matriz de Pruebas Funcionales, de Seguridad y Casos Límite

ID	Objetivo de la Prueba	Procedimiento Ejecutado	Resultado Obtenido
CP-01	Validación multicapa en registro (+18 años, DNI y email)	Envío de `POST /auth/registro` omitiendo validaciones HTML con DNI inválido o fecha de menor de 18 años.	**Exitoso:** Middleware y esquema rechazan con HTTP 400 mostrando alerta precisa sin persistir en BD.
CP-02	Bloqueo por fuerza bruta en login web y API	Ejecución de 6 peticiones consecutivas con clave errónea hacia `POST /auth/login` y `POST /auth/token`.	**Exitoso:** Al 6° intento responde `429 Too Many Requests` bloqueando temporalmente por 15 min.
CP-03	Autenticación JWT y rechazo de tokens inválidos/vencidos	Obtención de token en `/auth/token` y consumo de `GET /entradas` con token válido vs. token adulterado.	**Exitoso:** Con token válido retorna `200 OK`; adulterado o vencido devuelve `401` + header `WWW-Authenticate`.
CP-04	Control estricto de roles y protección de Superadmin	Intento de acceso con usuario `Lector` o `Cliente` a rutas de edición `/admin/salas` o intento de borrar a `admin`.	**Exitoso:** Responde página/JSON `403 Acceso Denegado`; el usuario `admin` permanece protegido en servidor.
CP-05	Compra online con descuento VIP y control de sobregiro	Compra con `cliente1` (VIP 30 %) superando y luego respetando el cupo disponible del sector VIP.	**Exitoso:** Rechaza exceso de cupo; con stock válido aplica 30 % OFF, crea `Transaccion` y emite QR.

ID	Objetivo de la Prueba	Procedimiento Ejecutado	Resultado Obtenido
CP-06	Validación única de QR y bloqueo de cancelación post-uso	Acreditación de ticket en `POST /admin/validar/:codigo`, segundo escaneo e intento posterior de cancelación.	**Exitoso:** Primer escaneo marca `Usado`; el reescaneo alerta ticket ya utilizado y cancelar devuelve HTTP 409.
CP-07	Integridad referencial y borrado lógico	Intento de eliminar una sala con eventos activos y posterior eliminación de un evento sin ventas conflictivas.	**Exitoso:** Sala con eventos devuelve HTTP 409; la baja marca `eliminado: true` conservando el histórico.
CP-08	Chat en tiempo real entre clientes y autoría segura	Dos clientes conectados; uno envía un mensaje indicando el nombre de otro usuario.	**Exitoso:** Llega al instante y el autor es el de la sesión. La empresa y las visitas no lo reciben.
CP-09	Consulta a Gemini y manejo de fallas	Mensajes con `@gemini`, sin prefijo y simulando un error 429 de la API. También se probó con la clave real.	**Exitoso:** Solo responde con el prefijo y a todos. Ante un error avisa y el servidor sigue activo.
CP-10	Privacidad y límites de las visitas	Visitantes con distinta dirección: 11 consultas de uno solo y carga simultánea de 100 visitas.	**Exitoso:** Nadie más ve la conversación. Corta en 10 por visita y 100 por hora, e invita a registrarse.
CP-11	Conteo de conectados por rol	Conexión de personal, clientes, visitas y varias pestañas, con desconexión posterior.	**Exitoso:** Cuenta a todos, una pestaña extra no suma y solo la empresa recibe el número.
CP-12	Estabilidad al cerrar sesión	25 cierres de sesión simultáneos con otras peticiones y sockets activos.	**Exitoso:** El servidor no se cae. Antes de la corrección, este caso lo detenía.

Pruebas automatizadas: Las pruebas están en la carpeta `tests/` y se ejecutan con `npm test`. Levantan el servidor real y verifican compras, roles, seguridad, informes, Excel, fotos, chat y límites, con 354 comprobaciones exitosas en la última ejecución. Las respuestas de Gemini se simulan para no consumir el cupo, y el funcionamiento con la clave real se verificó aparte. Se ejecutan siempre sobre una base local exclusiva de pruebas, de modo que nunca modifican los datos de MongoDB Atlas.

13 Historias de Usuario y Planificación

Las historias se priorizaron con el criterio Alta / Media / Baja y se estimaron por tamaño relativo (S, M o L). Este resumen se reconstruyó al cierre del proyecto a partir de las funcionalidades entregadas y del historial de GitHub.

ID	Historia de usuario	Prioridad	Tamaño	Estado
HU-01	Como visitante quiero ver la cartelera y los planes sin registrarme para decidir si me interesa.	Alta	M	Hecha
HU-02	Como visitante quiero registrarme e iniciar sesión para poder comprar.	Alta	M	Hecha

ID	Historia de usuario	Prioridad	Tamaño	Estado
HU-03	Como cliente quiero comprar entradas por sector para asegurar mi lugar.	Alta	L	Hecha
HU-04	Como cliente quiero contratar un plan para obtener descuentos.	Media	M	Hecha
HU-05	Como cliente quiero ver y descargar mis entradas con QR para ingresar al evento.	Alta	M	Hecha
HU-06	Como cliente quiero cancelar una entrada para liberar el lugar si no puedo asistir.	Media	M	Hecha
HU-07	Como supervisor quiero administrar salas, eventos y artistas para mantener la cartelera.	Alta	L	Hecha
HU-08	Como supervisor quiero vender en boletería para atender al público presencial.	Media	M	Hecha
HU-09	Como supervisor quiero validar el QR en la puerta para impedir que una entrada se use dos veces.	Alta	S	Hecha
HU-10	Como administrador quiero gestionar usuarios y roles para controlar los accesos.	Alta	M	Hecha
HU-11	Como lector quiero consultar informes y exportarlos a Excel para analizar las ventas.	Media	M	Hecha
HU-12	Como supervisor quiero auditar la actividad para saber qué se hizo en el sistema.	Media	M	Hecha
HU-13	Como cliente quiero un chat en vivo con asistente para resolver mis dudas al instante.	Media	M	Hecha
HU-14	Como visitante quiero consultar al asistente sin tener cuenta para aclarar mis dudas antes de registrarme.	Media	S	Hecha
HU-15	Como integrante de la empresa quiero ver cuántos usuarios hay conectados para conocer la actividad en vivo.	Baja	S	Hecha

Planificación por etapas (según el historial de GitHub)

Período	Etapa	Trabajo realizado
Septiembre 2026	1° Entrega	Estructura MVC, programación orientada a objetos, operaciones CRUD y validaciones iniciales.
Octubre 2026	2° Entrega	Persistencia con MongoDB, módulo de salas y zonas, informes, conexión a Atlas y filtros de búsqueda.
Noviembre 2026	3° Entrega	Nuevo diseño, sitio público, roles, despliegue en Render, Socket.IO, asistente Gemini, pruebas automáticas y documentación final.

Estado del tablero al cierre

Hecho	Pendiente para la entrega	Fuera de alcance
HU-01 a HU-15	Grabar el video de defensa y preparar los comprimidos individuales.	Pasarelas de pago reales y numeración individual de butacas por plano gráfico.

14 Mantenimiento, Evolución y Uso Consciente de Inteligencia Artificial

Mantenimiento correctivo y evolutivo: Respecto a la segunda entrega, se refactorizó por completo la capa de autenticación reemplazando el almacenamiento de sesiones en memoria (que perdía sesiones ante cada reinicio) por `connect-mongo` con expiración automática, y configurando `app.set('trust proxy', 1)` junto a una política de `Referrer-Policy` compatible para que las cookies seguras HTTPS y la verificación CSRF operen correctamente detrás del balanceador de Render. Para mejorar la **escalabilidad y el rendimiento**, se reemplazaron cálculos manuales en JavaScript por pipelines de agregación en MongoDB para los informes financieros, se ocultaron los buffers binarios de imágenes en listados generales mediante `select: false` y se incorporaron índices compuestos y paginación en la colección `Actividad`. En esta etapa final se corrigió además una falla que detenía el servidor cuando una sesión se cerraba mientras terminaba otra petición, y se actualizaron las librerías con vulnerabilidades conocidas (solo quedan dos de severidad moderada ligadas a `exceljs`, cuya corrección requiere un cambio incompatible de versión).

Declaración sobre el uso de Inteligencia Artificial: El desarrollo de esta entrega se realizó con un uso intensivo de herramientas de IA generativa, no como un apoyo puntual. Se utilizaron **Claude Code** (Anthropic) y **Google Antigravity** para generar y modificar código del servidor y de las vistas Pug, refactorizar, diagnosticar errores, escribir las pruebas automáticas y redactar, diseñar y revisar esta documentación, incluidos sus diagramas. Gran parte del código y del texto fue generado por estas herramientas a partir de las indicaciones del equipo, y luego adaptado y corregido en sucesivas iteraciones. El equipo definió el caso, los requerimientos, las prioridades y las decisiones funcionales (por ejemplo, qué puede ver cada rol), probó el sistema en funcionamiento y decidió qué cambios se aceptaban; el resultado se verificó con las pruebas automáticas y manuales documentadas. Dado que la IA puede equivocarse, cada integrante es responsable de revisar lo generado y de poder explicar la parte del sistema que le corresponde. Además, Gemini forma parte del producto como asistente del chat, tal como se explica en la sección correspondiente.

15 Conclusión Final del Cuatrimestre

✓ **¿Qué aprendimos este cuatrimestre?** Comprendimos cómo estructurar profesionalmente un servidor web en Node.js y Express aplicando separación de responsabilidades MVC, manejo centralizado de errores, asincronía con `async/await`, modelado documental en MongoDB Atlas y esquemas reales de seguridad (diferenciando cuándo aplicar sesiones de servidor con cookies seguras y cuándo utilizar autenticación *stateless* mediante JSON Web Tokens), llevando finalmente el producto a un entorno real en la nube, con comunicación en tiempo real mediante Socket.IO y un asistente basado en Gemini.

- **¿Qué dificultades tuvimos y cómo las resolvimos?** Los principales desafíos fueron tres: (1) compatibilizar las transacciones ACID entre MongoDB Atlas (clúster con replica set) y entornos locales *standalone* de desarrollo, lo cual resolvimos diseñando el utilitario `utils/transaccion.js` que detecta la topología al iniciar y aplica compensaciones automáticas en local; (2) lograr que las cookies seguras `secure: true` y la protección CSRF funcionaran detrás del proxy HTTPS en la nube ajustando `trust proxy` y las cabeceras de `helmet`; y (3) evitar que el crecimiento de la bitácora de auditoría degradara los tiempos de carga, resuelto con índices compuestos y consultas acotadas con `lean()`.

- **¿Qué parte del desarrollo Back End nos interesó más?** Nos resultó especialmente atractiva la arquitectura de seguridad y control de accesos por middlewares (combinar autenticación, roles jerárquicos, firma de tokens JWT y validaciones atómicas en la base de datos), así como ver reflejadas las reglas de negocio del mundo real —como evitar que dos compras simultáneas sobrevendan una sala o que un QR se utilice dos veces en la puerta— directamente en operaciones seguras del servidor.

- **¿Qué deberíamos reforzar a futuro?** De cara al Proyecto Final de la carrera, buscamos ampliar la cobertura de las pruebas automatizadas, la integración con pasarelas de pago reales y la automatización de pipelines CI/CD para despliegues continuos.